



UNIVERSITY OF MINNESOTA

Driven to Discover®

Audio

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Audio in Games

- Sound can make the difference between an immersive multimedia experience and a lackluster one
- The graphics engine reproduces what players **see** in the virtual world
- The audio engine reproduces what players **hear** in the virtual world
- Sound programmers use the term **audio rendering engine** to underscore parallels with the graphics engine

The Physics of Sound

Sound as a Compression Wave

- Sound is a **compression wave** that travels through air (or some other compressible medium)
- Sound pressure varies over time, so we plot it as a function $p_{inst}(t)$
- In **signal processing theory**, any time-varying function is called a **signal**

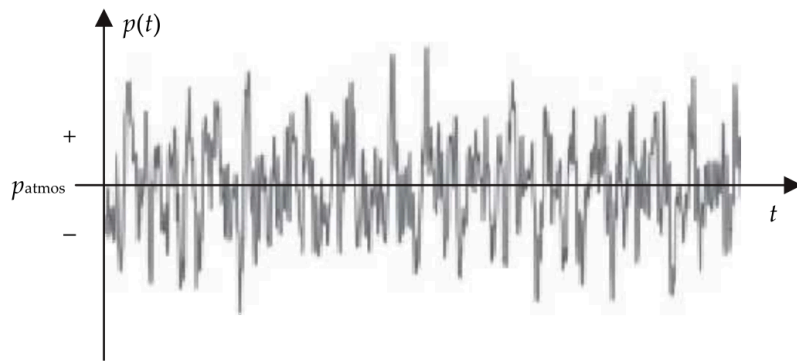


Figure 14.1. A signal $p(t)$ can be used to model the time-varying acoustic pressure of a sound.

Properties of Sound Waves

- A musical instrument's note produces a **periodic** signal: the waveform repeats
- The **period** T is the minimum time between successive repeats (seconds)
- The **frequency** is the inverse of the period: $f = 1/T$ (**Hertz**)

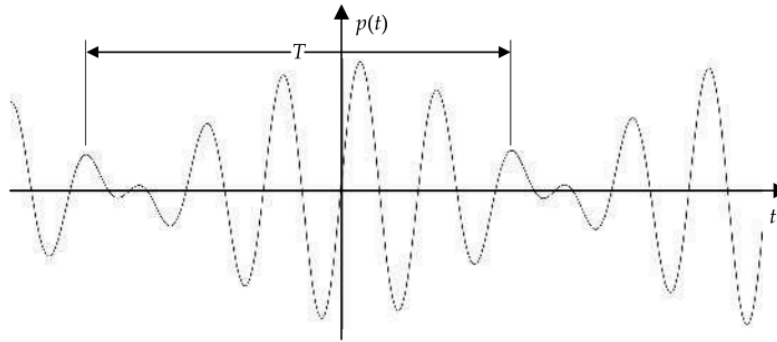


Figure 14.2. The period T of an arbitrary periodic signal is the minimum time between repeated patterns in the waveform.

Sound Wave Propagation

- Acoustic waves can be:
 - **absorbed** or **reflected** by surfaces
 - **diffracted** around corners and through narrow slits
 - **refracted** across the boundary of different transition media
- In games, we model absorption, reflection, and sometimes diffraction
- Refraction effects are typically ignored (not easily noticed)

Fall-Off with Distance

- For an omnidirectional source in open, still air:
 - **Pressure** falls off as $1/r$
 - **Intensity** falls off as $1/r^2$

$$I(r) \propto \frac{1}{r^2}$$

Sound Radiation Patterns

- **Omnidirectional**: radiates equally in all directions
- **Directional**: a flat reflective wall acts like this; wavefronts essentially parallel
- **Conical**: a bullhorn projects with maximum intensity along the centerline, falling off with angle



Figure 14.5. Three types of sound sources and their sound radiation patterns (in two dimensions for ease of illustration). From left to right: omnidirectional, conical and directional.

Perception of Position

- **Fall-off with distance** gives a sense of how far away the source is
 - Requires a "baseline" loudness to compare against
- **Atmospheric absorption** causes high frequencies to drop out with distance
 - Helps distinguish far/loud sounds from near/quiet sounds
- Two ears provide:
 - **Interaural time difference (ITD)**: ~1 ms delay between ears
 - **Interaural intensity difference (IID)**: head obstructs sound, the far ear gets a muffled version
- Ear shape muffles sounds from behind relative to those in front
 - Can be mathematically modeled using a **head-related transfer function (HRTF)**

Rendering Audio in 3D

3D Audio Rendering

- Any game in a 3D world needs a **3D audio rendering engine**
- Inputs: 3D sounds from the game world (footsteps, speech, gunfire, ambient sounds)
- Output: a handful of sound channels for the speakers
- Goal: rich, immersive, believable soundscape that matches the game world
- Should support 7.1 or 5.1 surround, but also stereo for headphones
- Also responsible for **2D sounds**: music, menu effects, narrator voice-overs
 - Played "directly" in the speakers, mixed with the 3D output

Tasks of the 3D Audio Engine

- **Sound synthesis**: generate the digital sound signals
 - Pre-recorded clips or procedurally generated at runtime
- **Spatialization**: produce the illusion of a sound's location
 - **Distance-based attenuation**: indicates radial distance
 - **Pan**: indicates direction
- **Acoustical modeling**: model early reflections, late reverberations, obstacles
- **Doppler shifting**: account for relative movement
- **Mixing**: control relative volumes of all 2D and 3D sounds

Modeling the Audio World

- **3D sound sources:**
 - Each consists of a monophonic signal at a specific position
 - Plus velocity, radiation pattern (omnidirectional, conical, planar), and range
- **Listener:**
 - A virtual microphone in the game world
 - Defined by position, velocity, and orientation
- **Environmental model:**
 - Geometry/properties of surfaces or acoustic properties of listening spaces

Distance-Based Attenuation

- Reduces the volume of a 3D sound as the radial distance from the listener increases
- A typical game world has many sound sources
 - We cannot render them all
 - Sounds beyond a certain distance cannot be heard anyway
- Each sound source is annotated with **fall-off (FO) parameters**

Fall-Off Min and Max

- **Fall-off min** (r_{min}):
 - minimum radius within which the sound is heard at full volume
- **Fall-off max** (r_{max}):
 - maximum radius beyond which the sound is silent
- Between FO min and FO max, blend smoothly from full volume to zero

Blending to Zero

- **Linear ramp** between FO min and FO max can sound just fine
- Using a **$1/r$ curve** is more physically accurate
 - Gain is proportional to amplitude
 - Problem: $1/r$ never quite reaches zero
- Two fixes:
 - Shift the curve down so it crosses the r axis at r_{max}
 - Clamp the intensity to zero for all $r > r_{max}$

Bending the Rules

- On **The Last of Us**, $1/r^2$ caused dialog to become unintelligible too quickly
 - A serious problem during stealth gameplay (hearing enemy chatter is critical)
- Solution: a sophisticated fall-off curve
 - Rolls off slowly near the listener
 - Faster in the mid-range
 - Slower again as distance grows large
- Fall-off curves were also adjusted dynamically based on the **tension level** of the game
- A "sweetener" reverb could let voices bleed around corners when the direct path is fully obstructed
- Lesson: do whatever it takes to satisfy the needs of the game

Atmospheric Attenuation

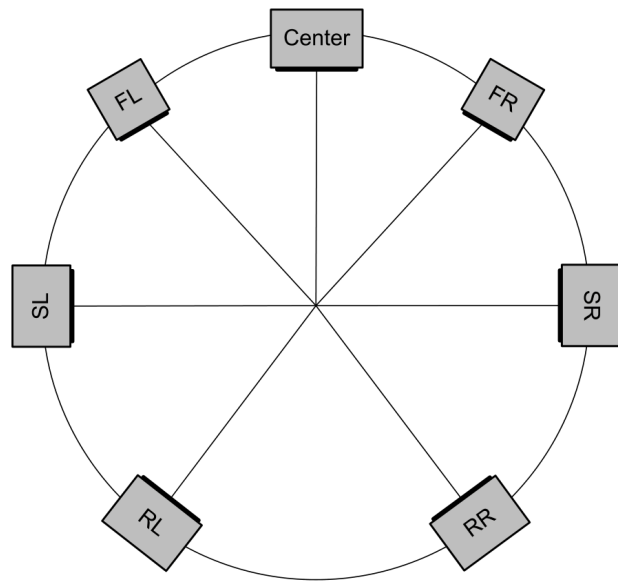
- Low-pitched sounds attenuate less than high-pitched sounds
- Some games (including The Last of Us) model this with a low-pass filter
- The passband slides toward lower frequencies as distance increases

Pan

- **Panning** provides the illusion that a 3D sound is coming from a particular direction
- Done by controlling the volume in each available speaker
 - Called **amplitude panning** (or **IID panning**)
 - Relies on interaural intensity difference
- Term comes from the early "panoramic potentiometer" or "pan pot"
- Panning can be visualized as a **speaker circle**
 - Listener is at the center of a circle
 - Speakers are positioned on the circumference

Pan Speaker Positions

- **Stereo speakers in a room:**
roughly ± 45 degrees
- **Stereo headphones:**
 ± 90 degrees (smaller radius)
- **7.1 surround:**
7 main speakers
- **5.1 surround:**
omit the SR and SL speakers



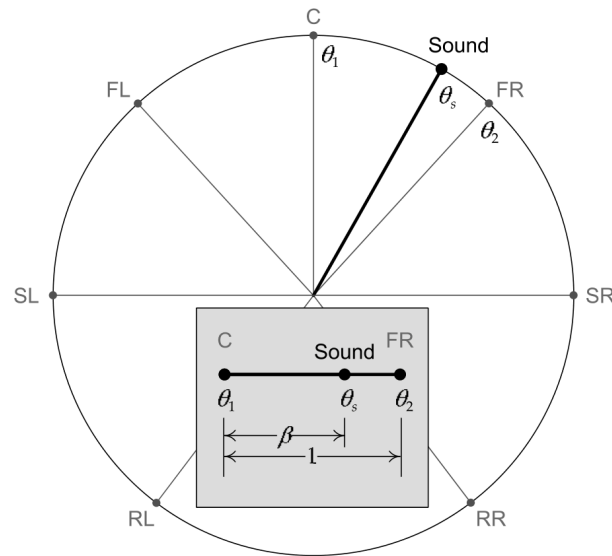
Speaker layout for 7.1 pan.

Computing the Pan Blend

- Treat each 3D sound as a point source
- Determine the **azimuthal angle** θ_s
(measured in listener's local space)
- Find the two speakers adjacent to that angle
(with angles θ_1 and θ_2)
- Compute the blend percentage:

$$\beta = \frac{\theta_s - \theta_1}{\theta_2 - \theta_1}$$

- Use β to determine the gains in each speaker



The pan blend percentage is calculated between two adjacent speakers.

Constant Gain Panning

- Linearly interpolate the gains between the two speakers:

$$A_1 = (1 - \beta)A, \quad A_2 = \beta A$$

- Net gain $A_1 + A_2$ is constant (hence "constant gain")
- **Problem:** does not produce the perception of constant **loudness**
 - Gain controls the amplitude of the sound pressure wave (SPL)
 - Human perception of loudness is proportional to intensity/power (square of the SPL)
- At the halfway point: $A_1 = A_2 = A/2$, total power is $\frac{1}{2}A^2$
 - The sound appears half as loud at the halfway point

The Constant Power Pan Law

- To keep loudness constant, we must keep **power** constant. This is the **pan law**.
- Use sine and cosine instead of linear interpolation:

$$A_1 = \sin\left(\frac{\pi}{2}\beta\right) A, \quad A_2 = \cos\left(\frac{\pi}{2}\beta\right) A$$

- At $\beta = \frac{1}{2}$: $A_1 = A_2 = \frac{1}{\sqrt{2}}A$, and total power is A^2 (constant).
- To account for the pan law, sound designers apply a **3 dB rule**.
 - When mixing equally to two speakers, reduce gain in each by 3 dB.

Headroom

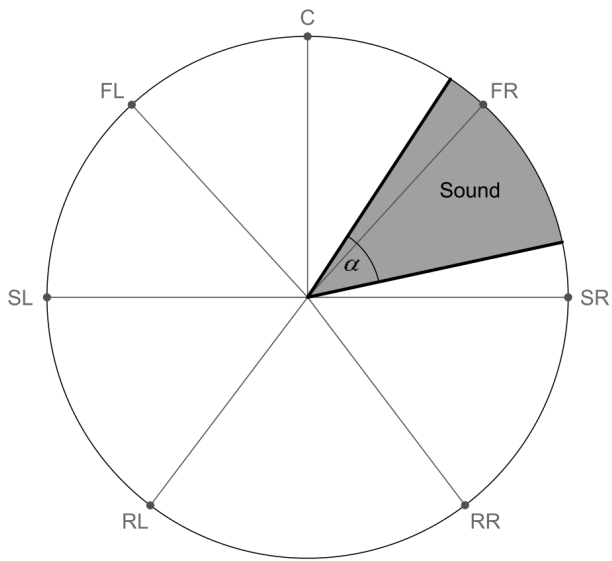
- Pan can result in a sound playing entirely from one speaker
- A sound at maximum volume across two speakers could overdrive a single speaker when panned
- We artificially lower maximum gains across the board to prevent overdrive
- This is called **leaving oneself some headroom**
- Headroom also matters for **mixing**: amplitudes add when sounds combine
 - Leaves room for worst-case scenarios where many loud sounds play together

To Center or Not to Center?

- In **cinema**, the center channel was historically used for speech
 - Characters are usually on-screen when speaking
 - Separates speech from sound effects, preserving headroom for dialog
- In **3D games**, players want dialog to come from the correct location
 - If the player rotates the camera 180°, dialog should rotate too
 - Games usually do not assign all dialog to the center
- Naughty Dog's compromise:
 - Split the dialog between the center and the panned speakers

Focus

- Far-away sources can be treated as **point sources**
- A sound very close to the listener cannot be modeled as a single point
- Moving past a sound source would cause it to "pop" from front to rear
- Instead, model an extended source as an **arc** on the speaker circle
- The arc subtends a **focus angle** α
- Sound is divided among speakers within (and adjacent to) the arc, obeying the constant power pan law



The focus angle defines the projection of an extended sound source on the speaker circle.

Dealing with Verticality

- Stereo and surround speakers all lie roughly in a horizontal plane
- Hard to position sounds above or below the listener's ears
- Ideal: a spherical speaker arrangement (**ambisonics**)
- **Trick:** project all sounds onto the horizontal plane
 - Use a focus angle for sounds whose projections fall too close
 - As an elevated sound passes overhead, blend across multiple speakers
- Combined with distance attenuation and atmospheric absorption, this provides enough cues for vertical perception

Propagation, Reverb, and Acoustics

- Distance attenuation and pan are not enough for a realistic soundscape
- Auditory cues for "what kind of space we're in" come from early reflections, late reverberations, and HRTF effects
- **Sound propagation modeling** accounts for ways sound waves propagate through space
- Three categories of techniques:
 - **Geometric analysis**: model the actual pathways of sound waves
 - **Perceptually based models**: reproduce what the ear perceives using LTI models
 - **Ad hoc methods**: approximations for reasonable accuracy with minimal cost

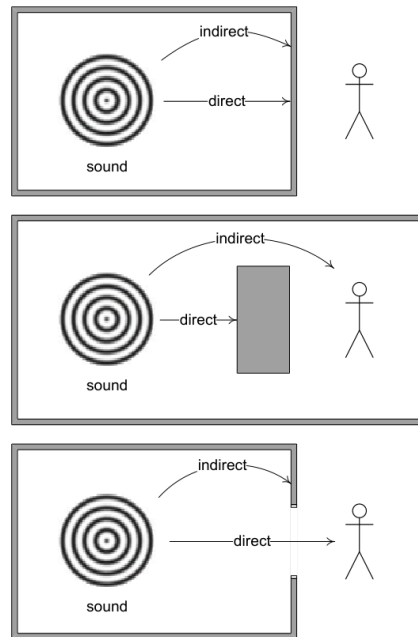
Reverb Regions

- Annotate the game world with manually placed regions
- Each region is tagged with reverb settings: pre-delay, decay, density, diffusion
- As the listener moves, light up the appropriate reverb mode
 - Large tiled room: bump up the echos
 - Small closet: dry sound
- Smoothly **cross-blend** between settings as the listener crosses regions
 - Linear interpolation per parameter
 - Blend percentage based on how far "into" the region the listener is

Obstruction, Occlusion, and Exclusion

A single set of reverb settings per region cannot capture acoustics caused by obstacles.

- **Occlusion:**
 - No unfettered path from source to listener
- **Obstruction:**
 - Direct path blocked, but indirect path available
- **Exclusion**
 - Free direct path, but the indirect path is compromised



From top to bottom: occlusion, obstruction and exclusion.

Analyzing the Direct Path

- Cast a ray from the listener to each sound source
 - If blocked: direct path is occluded
 - If free: direct path is unblocked
- To model sound through walls:
 - At each contact, query the surface's material properties
 - Determine how much energy is absorbed
 - Continue the ray on the other side
 - Attenuate sound by accumulated absorption

Analyzing the Indirect Path

- Determining indirect path occlusion is much harder
- An ideal A* path search is processor- and memory-intensive
- We are not chasing physical accuracy, just immersion
- Use heuristics with reverb regions:
 1. If source and listener are in the **same region**, assume the indirect path exists
 2. If they are in **different regions**, assume the indirect path is occluded

Accounting for Diffraction

- Waves spread out when they pass through narrow openings or interact with corners
- Sounds can be heard around corners as if a direct path existed

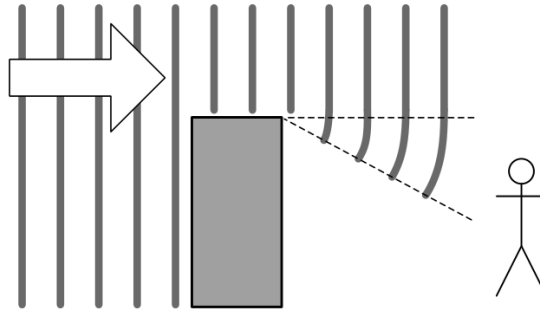


Figure 14.34. Diffraction causes the dry component of a sound to be clearly audible even when the direct path is blocked.

Accounting for Diffraction

- Cast a few **curved rays** around the central direct ray
 - Most engines do not support curved path tracing
 - Use multiple straight-line ray casts as a **piecewise-linear approximation**
- If the direct ray is occluded but curved traces reach the listener, the sound is in the diffraction region around a corner

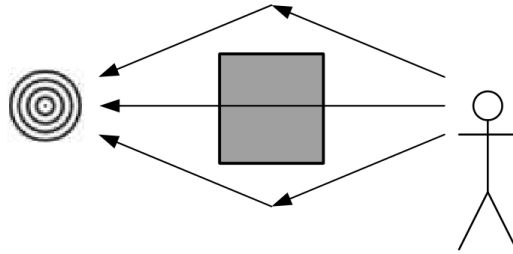


Figure 14.35. Curved ray casts can be approximated using multiple straight-line rays.

Sound Portals in The Last of Us

- Naughty Dog needed to model the actual paths sound takes through environments
- Sound from an NPC in a hallway should come from the doorway, not "through the wall"
- Solution: a network of interconnected regions
 - Two kinds: **rooms** and **portals**
- Same room: standard obstruction/occlusion/exclusion analysis
- Source in a connected room: play the sound as if it is located in the **portal region**
- Only one "hop" in the room connectivity graph was needed in practice

Sound Portals

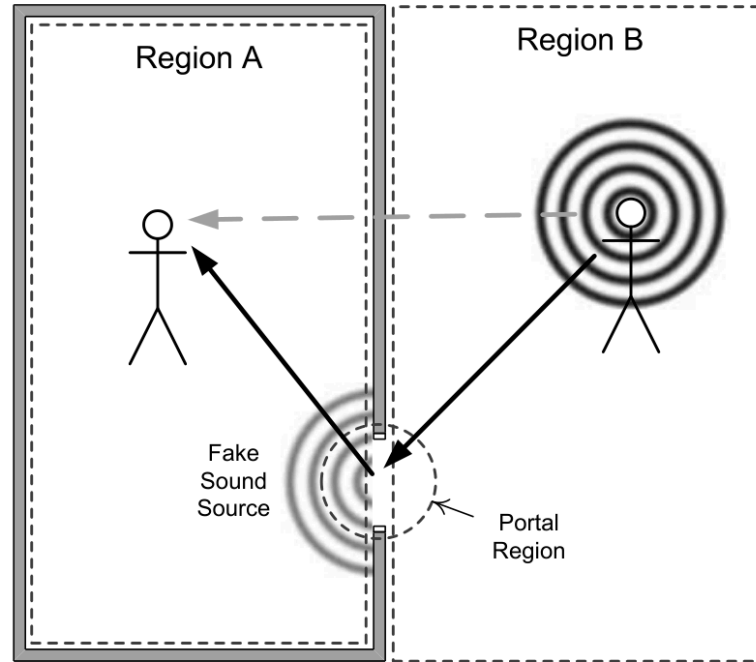


Figure I4.36. The portal-based audio propagation model used in *The Last of Us* by Naughty Dog, Inc.

Audio Engine Architecture

The Audio Hardware/Software Stack

- **Hardware:**

- May include codecs, mixers, reverb tanks, DSP chips

- **Driver layer** (on PC):

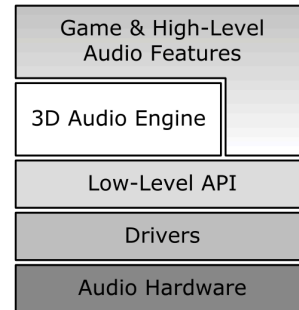
- Encapsulates hardware for OS support of various vendors

- **Low-level API:**

- Wraps hardware and drivers from controlling hardware directly

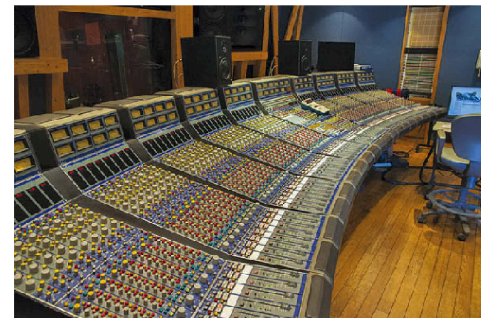
- **3D audio engine:**

- Built on top of these foundations



The Mixing Console Model

- Audio HW/SW stack feature set is modeled after a **multi-channel mixer console**
- Accepts many inputs from microphones and instruments
- Can be filtered, equalized, and processed with reverb
- Mixes signals together with desired relative volumes
- Final mix is routed to speakers or recording channels



Focusrite mixer console with support for 72 inputs and 48 outputs.

The Audio Processing Pipeline

Rendering a 3D sound involves several steps:

- For each 3D sound, synthesize a dry digital (PCM) signal
- Apply distance-based attenuation; apply reverb to produce a wet signal
- Pan the wet and dry signals (independently) to one or more speakers
- Mix all panned signals into a single multi-channel output
- Send through DACs and amps (analog) or directly to a digital output

The pipeline is sometimes called the **audio processing graph**.

The Audio Processing Graph

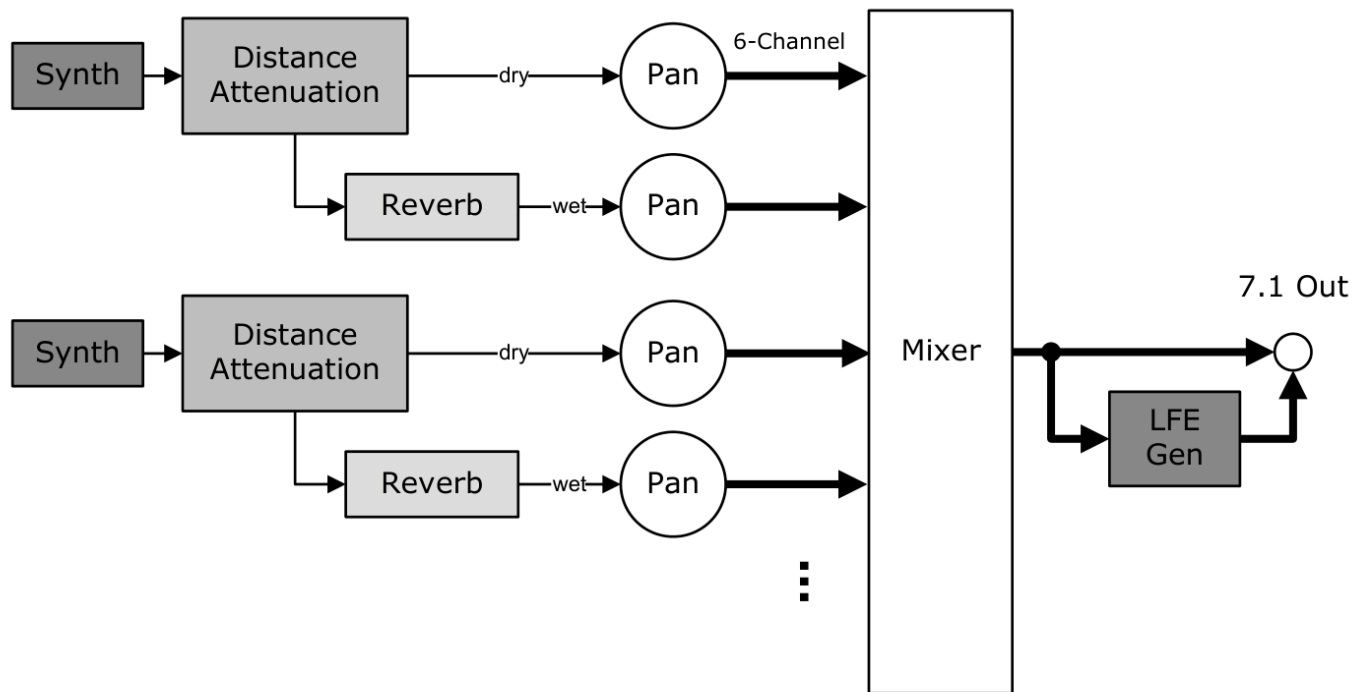


Figure 14.39. The audio processing graph (pipeline).

Asset Management: Audio Clips

- An **audio clip** is the most atomic audio asset
 - Single digital sound asset with its own local timeline
 - Analogous to an animation clip
- Sometimes called a **sound buffer**
- May encapsulate:
 - **Monophonic** audio (typical for 3D sound assets)
 - **Multi-channel** audio (typical for 2D assets or stereo 3D sources)
- Stored in any audio file format the engine supports

Sound Cues

- A **sound cue** is a collection of audio clips plus metadata describing how they should be processed and played
- Usually the primary means by which the game requests sound
 - Sound designers craft cues offline
 - Programmers play cues in response to game events
- A sound cue might:
 - Represent the channels of a pre-mixed 5.1 music recording
 - Collect raw sounds from which a random selection is made (for variety)
 - Play sounds in a predefined sequence
 - Specify whether it is a one-shot or looping sound

Sound Cue Metadata Examples

- Whether the cue is 2D or 3D
- FO min, FO max, and fall-off curve
- Group membership
- Special effects, filtering, equalization
- Optional **interruption sound clips**
 - (e.g., a glottal stop, or a "tail" for looping cues)
- A cue can even contain arbitrary script code that allows sound designer to control playback
 - Used by Sony's Scream Engine (sound engine used by Naughty Dog)

Playing a Cue

- The cue API is the primary (sometimes only) way for game code to play a sound
- Allows the programmer to specify:
 - 2D or 3D playback
 - 3D position and velocity
 - Loop or play once
 - In-memory or streamed
- Returns a **sound handle**
 - Lets the program track and modify a playing sound
 - Implemented as an index into a global handle table
 - Automatically nulled out when the sound ends
 - Provides thread safety: handles become invalid when killed

Sound Banks

- A 3D audio engine manages a lot of assets
- All audio data takes up an immense amount of space (cannot fit in memory at once)
- Individual clips are too fine-grained to manage individually
- Engines package clips and cues into coarse units called **sound banks**
- Some banks are loaded at startup and stay in memory (e.g., player character sounds)
- Others are loaded and unloaded dynamically as needs change
 - Last of Us: rain and creaking-beam sounds only loaded in the tilted Boston building

Streaming Sounds

- Some sounds are too long to store in memory all at once
 - Music, speech
- **Streaming audio** is possible because we only need data at and around the current time index
- Implementation:
 - Maintain a small **ring buffer** for each streaming sound
 - Pre-load a chunk before playing
 - Keep refilling the buffer ahead of playback

Mixing Your Game

If we played every sound from every object as physically modeled, the result would be **cacophony**.

The goal of a sound designer's mix:

- Sounds realistic and immersive
- Is not too distracting, annoying, or difficult to listen to
- Conveys all gameplay and story-relevant information effectively
- Maintains an appropriate mood and tonal color

Groups

- Set the levels of source sounds appropriately relative to one another
 - Footsteps should be quieter than gunfire
- Often we want to control an entire **category** of sounds at once
- A **group** is a category of sound cues whose gain can be controlled together
 - Borrowed from mixing consoles
- In software, groups are implemented by categorizing cues, not physically mixing them
- Allow categories to be paused, restarted, and muted with one API call

Ducking

- **Ducking**: temporarily reduce the volume of certain sounds to make others more audible
 - Reduce background noise during dialog
- Triggered by:
 - The presence of one type of sound
 - A game event (programmatic)
 - Any application-defined mechanism
- Typically implemented via the group categorization system

Instance Limiting

- Controls the number of sounds permitted to play simultaneously
- Two main reasons:
 - Prevents cacophony
 - Sound engines support a fixed number of voices
- Often applied differently per group:
 - Up to 4 guns simultaneously
 - Up to 3 people speaking
 - Up to 5 other sound effects
 - Up to 2 overlapping music tracks

Mixing In-Game Cinematics

- Normally the listener is at or near the camera
- During an **in-game cinematic**, the camera often pans away from the player
- Can wreak havoc with the 3D audio system
 - A long shot of two characters: physically too far away to be heard
- May want to detach the listener from the camera and reposition near the characters
- Mixing in-game cinematics is closer to mixing a film
 - Engine must "break the rules" when needed

Audio Engine Survey

- **Universal Audio Architecture (UAA)** on Windows
 - Standard since Vista
 - Limited HW features in driver, rest in software
 - Implemented via the **Windows Audio Session API (WASAPI)**
- **XAudio2**: high-powered low-level API for Xbox 360, Xbox One, and Windows
 - Replaced DirectAudio
 - **X3DAudio** is the 3D rendering library on top
- **Scream and BoomRangBuss**: Sony's 3D audio stack on PS3/PS4
 - Used by Naughty Dog
 - Mimics a multi-channel mixer console

Multiplatform 3D Audio Engines

- **OpenAL**: cross-platform 3D audio API designed to mimic OpenGL
 - Originally aimed to an open-source standard, eventually turned proprietary
 - Forked into OpenAL Soft, a widely used open-source software-only implementation
- **AeonWave 4D**: open-source, hardware-accelerated audio library for Windows and Linux
- **FMOD Studio**: audio authoring tool with a runtime 3D audio API
- **Miles Sound System**: popular middleware by Rad Game Tools
- **Wwise**: 3D audio engine by Audiokinetic, interface based on game objects/events
- **Unity** and **Unreal Engine** provide their own 3D audio engines and tool chains

Key Takeaways

- 3D audio rendering performs **synthesis, spatialization (distance attenuation and pan), acoustic modeling**, and **mixing**.
- The **constant power pan law** keeps perceived loudness constant as a sound's image moves between speakers.
- **Reverb regions, ray casts, diffraction approximations**, and **portals** model occlusion, obstruction, and exclusion.
- Asset management uses **clips, cues**, and **sound banks**, with streaming for long sounds.

Participation Exercise

Please submit the **student rating of teaching!**

There is a checkbox on Canvas to affirm you have submitted the SRT and get credit for today's participation exercise.



Questions?